T061 PS-014 BNPL Ledger: Week 1 and Week 2 Checkpoint Document

Oct 30, 2026 · @Lakshmi

Scope and checkpoint map

This document covers the first two checkpoints of PS-014: the Week 1 design items and the Week 2 core flows.

Item	Value
Team	T061 · C Vedasree (1AT24CG018), Chandan T (1AT24CG019) · Section CSD
Problem	PS-014 BNPL Pay-Later Ledger with Credit Line, Late Fees and Auto-Debit Retries
Language	Python 3.10+, standard library only (decimal, datetime, dataclasses, enum), pytest for tests
Repository	T061-PS-014

Checkpoint	Due	Required item	Section
Week 1 – Design	Fri, 23 Oct 2026	Glossary	1
Week 1 – Design	Fri, 23 Oct 2026	Class diagram	2
Week 1 – Design	Fri, 23 Oct 2026	State diagram	3
Week 1 – Design	Fri, 23 Oct 2026	Business-rule register (BR-01 …)	4
Week 2 – Core flows	Fri, 30 Oct 2026	Git repo shared with organisers	5
Week 2 – Core flows	Fri, 30 Oct 2026	Happy path working on sample data	6
Week 2 – Core flows	Fri, 30 Oct 2026	Unit tests for main rules	7

1. Glossary

Every term below is used with exactly this meaning in the code, the tests and the output files.

Term	Meaning in this engine
Allocation	The order in which a payment fills a user's dues: bounce charges, late fees, convenience fee, its GST, then principal, oldest bill first within each (BR-19)
Auto-debit	A scheduled pull from the user's bank under the mandate. Attempt 1 on the due date 08:00, retries at due + 2 and due + 5 days (BR-14)
Available line	Current line minus unbilled net purchases, unpaid bill principal and unpaid fees. Recomputed on every check, never stored (BR-06)
Bill	The statement for one user and one billing cycle. Holds principal, convenience fee, GST, late fee and bounce charges as separate components
Bill date	The cycle end date (15th or month end). The bill is generated at 00:00:00 on the next day (BR-09)
Billing cycle	One of two periods a month: C1 = 1st to 15th, C2 = 16th to month end. ID format 2027-01-C1 (BR-08)
Bounce charge	Rs 50 added to a bill for each failed debit attempt after the first (BR-15)
Bureau file	Month-end report of every user's state, reported amount and DPD, written at 00:00:00 on the 1st (BR-25, BR-26)
Convenience fee	1% of the bill when the bill is above Rs 1,000, capped at Rs 100. Zero otherwise (BR-10)
Credit balance	Money owed to the user from a post-billing refund or an overpayment. Applied to the next bill (BR-12, A-15)
Current line	The line used for limits today. Equals the original line until HARD_FREEZE reduces it
Decision	APPROVED or DECLINED for a PURCHASE or REFUND, with every failed rule ID (A-17)
DPD	Days past due. Report date minus the earliest unpaid due date, 0 if nothing is overdue (BR-25)
Due date	Bill date + 5 days (BR-09)

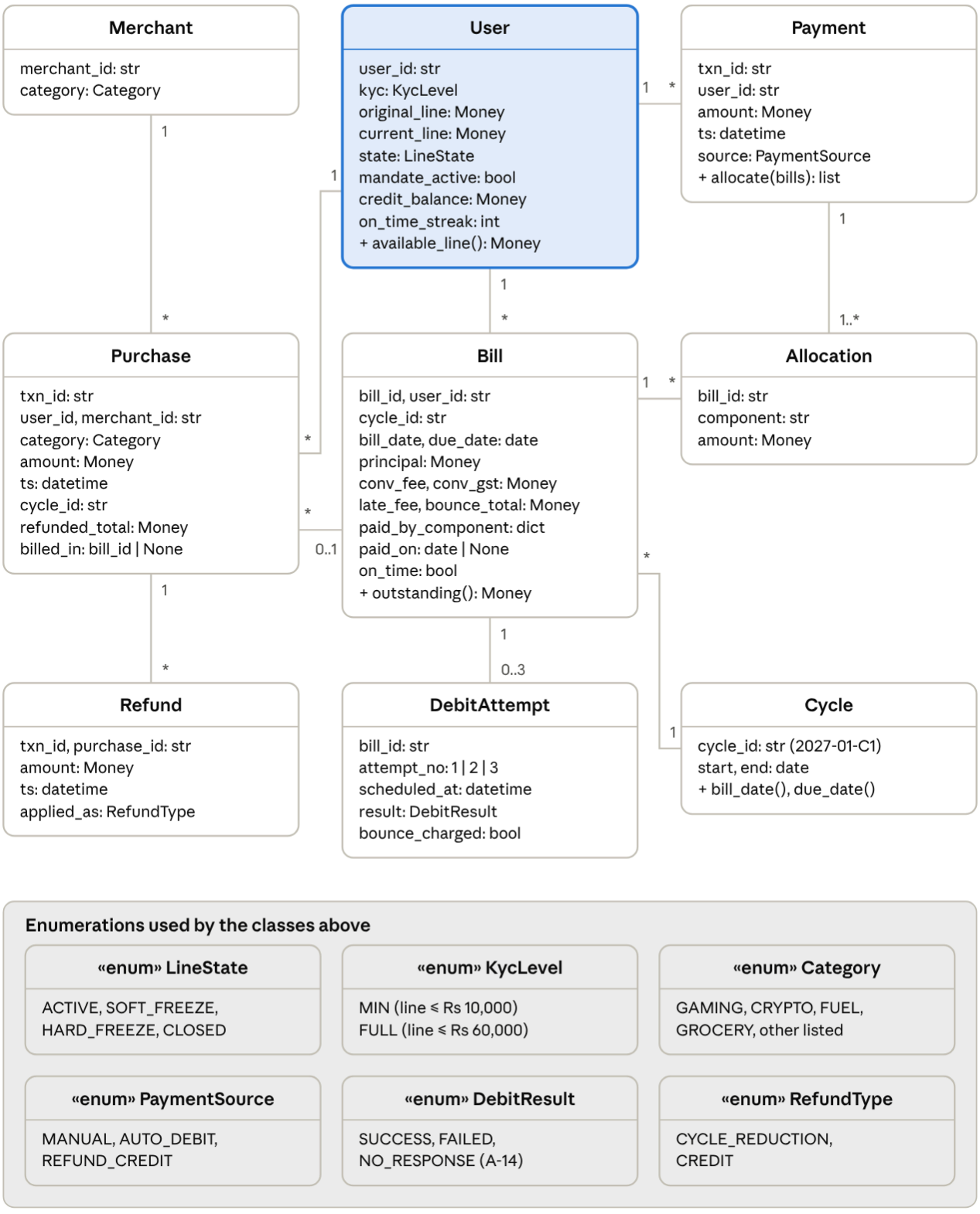
Term	Meaning in this engine
GST	18% tax on the convenience fee only. Excluded from the bureau amount (A-03, BR-26)
KYC level	MIN (line up to Rs 10,000) or FULL (line up to Rs 60,000) (BR-01)
Late fee	Rs 50, Rs 150 or Rs 300 by bill size, charged once at due + 6 days 00:00:00, capped at 10% of the bill (BR-17, BR-18)
Line state	ACTIVE, SOFT_FREEZE, HARD_FREEZE or CLOSED. Controls whether purchases are allowed (BR-07, BR-20 to BR-24)
Mandate	The user's standing permission for auto-debit. Once cancelled, no further attempts run (BR-16)
Merchant category	GAMING, CRYPTO, FUEL, GROCERY or another listed category. Drives the category rules (BR-03 to BR-05)
On-time bill	A bill fully paid by its due date, including one paid only by refund credit (A-07)
Original line	The sanctioned line from onboarding. Restored after 3 consecutive on-time bills (BR-24)
Principal	Net purchases billed in a cycle after refunds and credit, excluding fees
Rule ID	Short code printed on a decline or rejection: O01, P01, M01, M02, L01, S01 to S03, R01, R02, V01 to V05
Unbilled net	Approved purchases in the open cycle minus refunds against them

2. Class diagram

Nine domain classes live in models.py. A User owns its purchases, bills and payments, and each Bill keeps its fee components separate so allocation and fee rules can read exactly the part they need.

A user owns purchases, bills and payments; each bill holds its own fee parts

Domain classes in models.py · lines are associations, numbers are multiplicities



Enumerations used by the classes above

«enum» LineState	«enum» KycLevel	«enum» Category
ACTIVE, SOFT_FREEZE, HARD_FREEZE, CLOSED	MIN (line ≤ Rs 10,000) FULL (line ≤ Rs 60,000)	GAMING, CRYPTO, FUEL, GROCERY, other listed

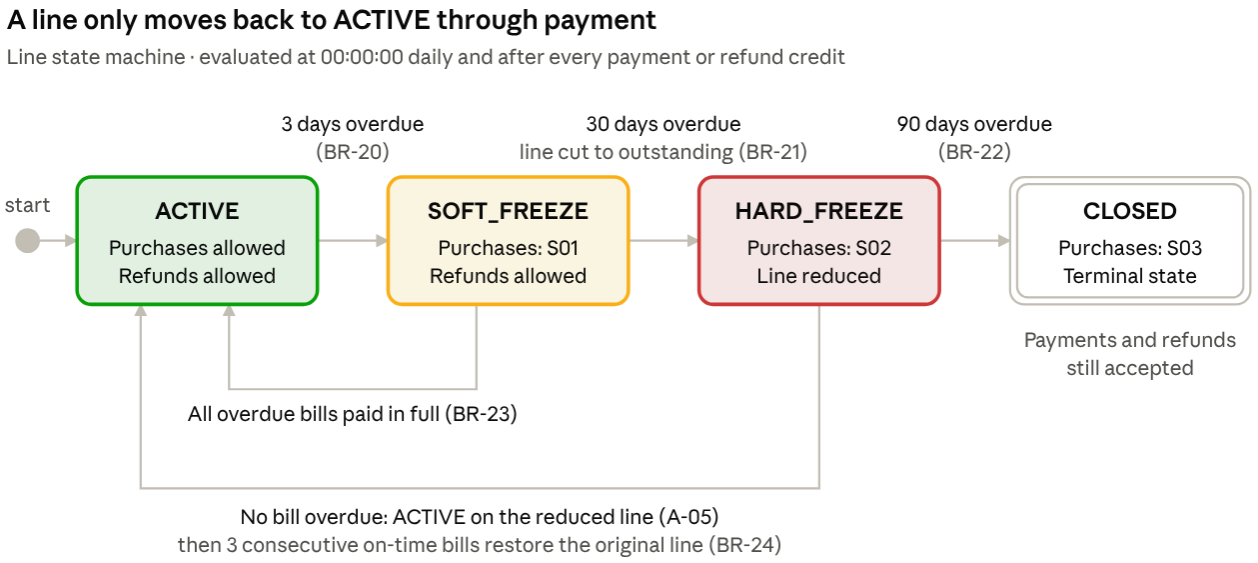
«enum» PaymentSource	«enum» DebitResult	«enum» RefundType
MANUAL, AUTO_DEBIT, REFUND_CREDIT	SUCCESS, FAILED, NO_RESPONSE (A-14)	CYCLE_REDUCTION, CREDIT

class diagram · 9 domain classes, 6 enumerations

Payment and Bill are many-to-many through Allocation (BR-19). Available line is computed by User.available_line() from these objects on every check, never stored (BR-06). Every money field uses the Money type: Decimal, 2 places, ROUND_HALF_UP.

3. State diagram

Every credit line is in exactly one of four states. It moves right as its earliest unpaid bill ages, and returns to ACTIVE only through payment.



line state machine · 4 states, 5 transitions

From	To	Trigger	Action
(start)	ACTIVE	User onboarded (BR-01)	Current line = original line
ACTIVE	SOFT_FREEZE	Earliest unpaid bill 3 days overdue (BR-20)	Purchases declined with S01
SOFT_FREEZE	HARD_FREEZE	30 days overdue (BR-21)	Current line set to total outstanding, so available line = 0; original line kept for restoration
HARD_FREEZE	CLOSED	90 days overdue (BR-22)	Terminal; payments and refunds still accepted
SOFT_FREEZE	ACTIVE	Every overdue bill fully paid, fees included (BR-23, A-06)	Restored immediately
HARD_FREEZE	ACTIVE	No bill overdue (A-05)	Line stays reduced; on-time streak starts, and 3 consecutive on-time bills restore the original line (BR-24)

Days overdue = current date − earliest due date among unpaid bills. If input dates skip ahead, the engine still logs each step it passes through.

4. Business rule register

The engine implements 26 business rules (BR-01 to BR-26) and 5 validation rules (V01 to V05). The Output ID is what a decision line prints when the rule declines something.

BR	Rule	Output ID	Module
BR-01	Line at onboarding: MIN KYC ⩽ Rs 10,000; FULL KYC ⩽ Rs 60,000	O01	users.py
BR-02	Single purchase ⩽ min(50% of current line, Rs 15,000); GROCERY exempt (A-08)	P01	authorisation.py
BR-03	GAMING and CRYPTO purchases blocked	M01	authorisation.py
BR-04	FUEL net purchases ⩽ Rs 2,000 per billing cycle	M02	authorisation.py
BR-05	GROCERY allowed up to 100% of available line	none	authorisation.py
BR-06	Available line = line − unbilled net purchases − unpaid bill principal − unpaid fees; decline if result < 0	L01	ledger.py
BR-07	Purchases allowed only in ACTIVE; refunds allowed in every state	S01 / S02 / S03	authorisation.py
BR-08	Two cycles a month: 1st–15th and 16th–month end	none	calendar.py
BR-09	Bill dated on cycle end date; due date = bill date + 5 days	none	billing.py
BR-10	Convenience fee: 0 if bill ⩽ Rs 1,000; else 1% of bill, max Rs 100; plus 18% GST on the fee	none	billing.py
BR-11	Refund of a purchase in the open cycle reduces that cycle's bill	none	billing.py
BR-12	Refund of an already billed purchase becomes credit on the next bill (A-09)	none	billing.py

BR	Rule	Output ID	Module
BR-13	Cumulative refunds on a purchase ⩽ purchase amount; refund must reference a known purchase of the same user	R01 / R02	authorisation.py
BR-14	Auto-debit at due date 08:00; retries at due + 2 days and due + 5 days, 08:00	none	autodebit.py
BR-15	Each failed attempt after the first adds a Rs 50 bounce charge	none	autodebit.py
BR-16	No attempt or retry once the mandate is cancelled	none	autodebit.py
BR-17	Late fee if bill unpaid after due + 5 days: Rs 50 (⩽ 2,000), Rs 150 (⩽ 5,000), Rs 300 (above)	none	fees.py
BR-18	Late fee capped at 10% of bill amount	none	fees.py
BR-19	Payment allocation: bounce → late fee → convenience fee (+GST) → principal, oldest bill first within each	none	payments.py
BR-20	SOFT_FREEZE when any bill is 3 days overdue	none	state_machine.py
BR-21	HARD_FREEZE at 30 days overdue; line reduced to current outstanding	none	state_machine.py
BR-22	CLOSED at 90 days overdue (terminal)	none	state_machine.py
BR-23	Full payment of overdue bills in SOFT_FREEZE restores ACTIVE immediately	none	state_machine.py
BR-24	After HARD_FREEZE, 3 consecutive on-time bills restore the original line	none	state_machine.py
BR-25	Bureau DPD = days past the earliest unpaid due date; 0 if nothing overdue	none	bureau.py
BR-26	Bureau amount excludes GST on fees	none	bureau.py

ID	Validation rule	Checked in
V01	Unknown user	io/parser.py, authorisation.py

ID	Validation rule	Checked in
V02	Amount ⩽ 0 or more than 2 decimal places	io/parser.py
V03	Unknown merchant or unknown category	io/parser.py
V04	Duplicate transaction ID	authorisation.py
V05	Malformed line or timestamp earlier than the previous command (input must be time-ordered)	io/parser.py

5. Git repository

The code lives in a GitHub repository named T061-PS-014, and its link is sent to support@10seconds.co.in on the existing team thread by the Week 2 checkpoint. All later checkpoints are read from this repository.

Item	Value
Repository name	T061-PS-014
Link	To add once created: https://github.com/<owner>/T061-PS-014
Shared with	support@10seconds.co.in, reply on the same email thread
Default branch	main, always green (all tests pass)
Working branches	feature/<module>, merged to main after tests pass

Layout

```
T061-PS-014/
├── README.md                setup, run command, folder guide
├── docs/
│   └── design_document.pdf   Week 1 design
├── bnpl/
│   ├── money.py             Decimal wrapper, ROUND_HALF_UP to 0.01
│   ├── calendar.py          cycle IDs, cycle end, due dates
│   ├── rules.py             constants, assumption switches, rule IDs
│   ├── models.py            dataclasses, LineState and other enums
│   ├── ledger.py            available line, outstanding by component
│   ├── users.py             onboarding (BR-01)
│   ├── authorisation.py     purchase and refund checks
│   ├── billing.py           bill generation, convenience fee, GST
```

```
|   ├── payments.py           allocation, credit balance
|   ├── autodebit.py          attempt schedule, success posting, skips
|   ├── engine.py             day loop and event ordering
|   ├── io/parser.py          input parsing with V-rule validation
|   ├── io/writer.py          CSV outputs
|   └── cli.py                command-line entry point
├── data/
|   ├── sample/               organisers' sample files (as received)
|   └── happy_path/           the small scenario in section 6
└── tests/
    ├── unit/                 test_br_NN_*.py, test_v0N_*.py
    └── scenarios/            test_happy_path.py
```

Setup and run

```
git clone https://github.com/<owner>/T061-PS-014.git
cd T061-PS-014
python -m venv .venv && source .venv/bin/activate
pip install pytest

python -m bnpl run --users data/sample/users.csv --merchants
data/sample/merchants.csv \
  --commands data/sample/commands.csv --debits data/sample/debit_results.csv
--out out/

pytest -q
```

The run writes decisions.csv, bills.csv and one ledger print per user into the out/ folder.

6. Happy path: core flow on sample data

The happy path takes a user from onboarding through purchases, a refund, bill generation and a successful auto-debit back to a fully available line, with the line ACTIVE throughout. It is first proven on the small scenario below, then run on the organisers' sample files.

Order of processing for each simulated day D

1. 00:00:00: if D is the 16th or the 1st, generate bills for the cycle that just ended.
2. 08:00:00: run scheduled debit attempts and read their results.
3. Run all commands for D in timestamp order. System events at an instant run before commands at the same instant.

Scenario input (data/happy_path/)

```
user_id,kyc,line,mandate,onboard_date
U01,FULL,20000.00,ACTIVE,2027-01-01
U02,MIN,8000.00,ACTIVE,2027-01-01
```

```
merchant_id,category
M01,ELECTRONICS
M02,GROCERY
M03,FUEL
```

```
ts,type,txn_id,user_id,merchant_id,amount,ref_txn_id
2027-01-03T10:15:00,PURCHASE,T0001,U01,M01,4000.00,
2027-01-05T12:00:00,PURCHASE,T0002,U01,M03,1500.00,
2027-01-09T18:40:00,REFUND,T0003,U01,M01,500.00,T0001
2027-01-12T09:00:00,PURCHASE,T0004,U01,M02,2000.00,
2027-01-14T16:30:00,PURCHASE,T0005,U02,M02,800.00,
2027-01-18T11:00:00,PAYMENT,T0006,U02,,800.00,
```

```
ts,user_id,bill_id,attempt_no,result,amount
2027-01-20T08:00:00,U01,B0001,1,SUCCESS,7082.60
```

Walkthrough

When	Event	Rules applied	Result	U01 available (Rs)
1 Jan	Onboard U01 (FULL, 20,000) and U02 (MIN, 8,000)	BR-01	Both accepted, ACTIVE, cycle 2027-01-C1	20,000.00
3 Jan 10:15	T0001 purchase 4,000	BR-02 (cap 10,000), BR-06	APPROVED	16,000.00
5 Jan 12:00	T0002 FUEL purchase 1,500	BR-04 (1,500 ⩽ 2,000)	APPROVED	14,500.00
9 Jan 18:40	T0003 refund 500 of T0001	BR-13, BR-11	APPROVED, reduces open cycle	15,000.00
12 Jan 09:00	T0004 GROCERY purchase 2,000	BR-05	APPROVED	13,000.00
14 Jan 16:30	T0005 U02 GROCERY purchase 800	BR-05	APPROVED, U02 available 7,200.00	13,000.00

When	Event	Rules applied	Result	U01 available (Rs)
16 Jan 00:00	Bill B0001 for U01: principal 7,000.00, fee 70.00, GST 12.60	BR-08, BR-09, BR-10	Total 7,082.60, due 20 Jan	12,917.40
16 Jan 00:00	Bill B0002 for U02: principal 800.00, no fee	BR-10	Total 800.00, due 20 Jan	12,917.40
18 Jan 11:00	T0006 U02 manual payment 800	BR-19	B0002 paid on time, U02 available 8,000.00	12,917.40
20 Jan 08:00	Attempt 1 on B0001 succeeds for 7,082.60	BR-14, BR-19	Fee 70.00, GST 12.60, principal 7,000.00 cleared; paid on time	20,000.00
20 Jan 08:00	Attempt 1 on B0002	BR-14 skip rule	Skipped, bill already paid, no charge	20,000.00
1 Feb 00:00	C2 cycle closes with no activity	A-07	No C2 bill created	20,000.00

Expected output

```
ts,txn_id,user_id,type,amount,decision,rule_ids,available_after
2027-01-03T10:15:00,T0001,U01,PURCHASE,4000.00,APPROVED,,16000.00
2027-01-05T12:00:00,T0002,U01,PURCHASE,1500.00,APPROVED,,14500.00
2027-01-09T18:40:00,T0003,U01,REFUND,500.00,APPROVED,,15000.00
2027-01-12T09:00:00,T0004,U01,PURCHASE,2000.00,APPROVED,,13000.00
2027-01-14T16:30:00,T0005,U02,PURCHASE,800.00,APPROVED,,7200.00
```

```
bill_id,user_id,cycle_id,bill_date,due_date,principal,conv_fee,gst,late_fee,b
ounce,total,paid_on,status
B0001,U01,2027-01-C1,2027-01-15,2027-01-
20,7000.00,70.00,12.60,0.00,0.00,7082.60,2027-01-20,PAID_ON_TIME
B0002,U02,2027-01-C1,2027-01-15,2027-01-
20,800.00,0.00,0.00,0.00,0.00,800.00,2027-01-18,PAID_ON_TIME
```

The scenario is locked in by tests/scenarios/test_happy_path.py, which diffs decisions.csv and bills.csv line by line.

Run on the organisers' sample data

The input formats above are provisional and are aligned to the sample file headers when they arrive.

- [ ] Copy the sample files into data/sample/ unchanged
- [ ] Align parser column names and order to the sample headers
- [ ] Run the full 200-command file end to end with no crash
- [ ] Diff decisions and bills for the 7 visible users against the organisers' visible expected outputs

7. Unit tests for main rules

The unit tests cover the rules the happy path depends on: onboarding, authorisation, available line, billing, refunds, the debit schedule, payment allocation and input validation. Each test uses boundary values, so a one-paisa error fails it.

Test	Rule	Input	Expected
test_money_rounding	Money	10.005 × 1	10.01 (ROUND_HALF_UP)
test_br_01_onboarding_line_limit	BR-01	MIN 10,000.00 / 10,000.01; FULL 60,000.00 / 60,000.01	Accepted / O01; accepted / O01
test_br_02_single_purchase_cap	BR-02	Line 20,000: 10,000.00 then 10,000.01. Line 40,000: 15,000.01	APPROVED, P01; P01 (Rs 15,000 ceiling)
test_br_03_blocked_categories	BR-03	GAMING 100.00; CRYPTO 100.00	M01; M01
test_br_04_fuel_cycle_cap	BR-04	FUEL 1,500.00, 500.00, then 0.01 in one cycle	APPROVED, APPROVED, M02
test_br_05_grocery_full_line	BR-05	Line 8,000: GROCERY 8,000.00, then 0.01	APPROVED with available 0.00 (P01 skipped), then L01
test_br_06_available_line	BR-06	Line 10,000; unpaid bill 3,000 + 30.00 + 5.40; unbilled 2,500 − 500	Available 4,964.60; 4,964.60 APPROVED, 4,964.61 L01

Test	Rule	Input	Expected
test_br_07_purchase_only_active	BR-07	Purchase in SOFT_FREEZE / HARD_FREEZE / CLOSED; refund in CLOSED	S01 / S02 / S03; refund APPROVED
test_br_08_cycle_boundary	BR-08	2027-01-15T23:59:59; 2027-01-16T00:00:00	2027-01-C1; 2027-01-C2
test_br_09_due_date	BR-09	Bill dates 15 Jan, 31 Jan, 28 Feb 2027	Due 20 Jan, 5 Feb, 5 Mar
test_br_10_convenience_fee	BR-10	Principal 1,000.00 / 1,000.01 / 5,000.00 / 12,000.00	Fee + GST 0 + 0 / 10.00 + 1.80 / 50.00 + 9.00 / 100.00 + 18.00
test_br_11_refund_open_cycle	BR-11	C1 purchase 4,000, C1 refund 500	C1 bill principal 3,500.00
test_br_12_refund_after_billing	BR-12	C1 bill 4,000; refund 500 on 20 Jan; C2 purchase 1,000	Credit 500.00; C2 bill principal 500.00
test_br_13_refund_limits	BR-13	Refunds 2,500.00 then 1,500.01 on a 4,000 purchase; refund on another user's purchase	APPROVED, R01; R02
test_br_14_debit_schedule	BR-14	Due 2027-01-20	Attempts 20 Jan, 22 Jan, 25 Jan at 08:00
test_br_19_allocation	BR-19	B1 owes bounce 50, late 150, fee 20.00, GST 3.60, principal 2,000; B2 principal 1,000; pay 300	B1: 50, 150, 20.00, 3.60, principal 76.40; B2 untouched
test_br_19_overpayment_credit	BR-19, A-15	Bill total 800.00, pay 1,000.00	Bill paid; credit balance 200.00
test_v01_unknown_user	V01	Purchase by U99	V01
test_v02_amount	V02	Amounts 0.00, −5.00, 10.001	V02 each; no state change

Test	Rule	Input	Expected
test_v04_duplicate_txn	V04	T0001 sent twice	Second is V04
test_v05_time_order	V05	Command stamped before the previous one	V05
test_happy_path	Section 6	data/happy_path/	decisions.csv and bills.csv match line by line

Sample tests

```
from decimal import Decimal as D
import pytest
from bnpl.billing import convenience_fee
from bnpl.ledger import available_line
from bnpl.authorisation import authorise_purchase

@pytest.mark.parametrize("principal, fee, gst", [
    ("1000.00", "0.00", "0.00"),
    ("1000.01", "10.00", "1.80"),
    ("5000.00", "50.00", "9.00"),
    ("12000.00", "100.00", "18.00"),
])
def test_br_10_convenience_fee(principal, fee, gst):
    assert convenience_fee(D(principal)) == (D(fee), D(gst))

def test_br_06_available_line(make_user):
    u = make_user(line="10000.00")
    u.add_unpaid_bill(principal="3000.00", conv_fee="30.00", gst="5.40")
    u.purchase("2500.00")
    u.refund("500.00")
    assert available_line(u) == D("4964.60")
    assert authorise_purchase(u, D("4964.61")).rule_ids == ["L01"]
```

Run

```
pytest -q                      # whole suite
pytest tests/unit -k br_10     # one rule
pytest tests/scenarios         # happy path
```